SINGLE CLIENT CHATTING - TCP AND UDP COMMUNICATION IN NS-3

MUTHU VENKATESH M

24BCE1389

Lab Task: Implement a **single-client chatting application** using both **TCP and UDP protocols**.

CODE:

```
#include "ns3/core-module.h"

#include "ns3/network-module.h"

#include "ns3/internet-module.h"

#include "ns3/point-to-point-module.h"

#include "ns3/applications-module.h"


using namespace ns3;


void ReceivePacket (Ptr<Socket> socket)

{

 Ptr<Packet> packet;

 Address from;


 while ((packet = socket->RecvFrom (from)))

 {

  std::cout << "Server received "

  << packet->GetSize ()
```

```
  << " bytes at "
  << Simulator::Now ().GetSeconds ()
  << " s\n";
 }
}

void SendPacket (Ptr<Socket> socket,
                uint32_t pktSize,
                uint32_t pktCount,
                Time pktInterval)
{
 if (pktCount > 0)
 {
  socket->Send (Create<Packet> (pktSize));

  Simulator::Schedule (pktInterval,
                    &SendPacket,
                    socket,
                    pktSize,
                    pktCount - 1,
                    pktInterval);
 }
 else
 {
  socket->Close ();
 }
}
```

```
int main (int argc, char *argv[])
{
 Time::SetResolution (Time::NS);

 // Create nodes
 NodeContainer nodes;
 nodes.Create (2);

 // Point-to-point link
 PointToPointHelper p2p;
 p2p.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
 p2p.SetChannelAttribute ("Delay", StringValue ("2ms"));

 NetDeviceContainer devices = p2p.Install (nodes);

 // Install internet stack
 InternetStackHelper internet;
 internet.Install (nodes);

 // Assign IP addresses
 Ipv4AddressHelper ipv4;
 ipv4.SetBase ("10.1.1.0", "255.255.255.0");

 Ipv4InterfaceContainer interfaces =
 ipv4.Assign (devices);

 uint16_t port = 8080;
```

```
// ---------------- SERVER ----------------

Ptr<Socket> serverSocket =
Socket::CreateSocket (nodes.Get (1),
UdpSocketFactory::GetTypeId ());

InetSocketAddress local =
InetSocketAddress (Ipv4Address::GetAny (), port);

serverSocket->Bind (local);

serverSocket->SetRecvCallback
(MakeCallback (&ReceivePacket));

// ---------------- CLIENT ----------------

Ptr<Socket> clientSocket =
Socket::CreateSocket (nodes.Get (0),
UdpSocketFactory::GetTypeId ());

InetSocketAddress remote =
InetSocketAddress (interfaces.GetAddress (1), port);

clientSocket->Connect (remote);

Simulator::Schedule (Seconds (2.0),
&SendPacket,
clientSocket,
```

```
1024,       // packet size
10,         // number of packets
Seconds (1.0)); // interval

Simulator::Stop (Seconds (15.0));

Simulator::Run ();
Simulator::Destroy ();

return 0;
}
```

OUTPUT:

```
Server received 1024 bytes at 2.00369 s
Server received 1024 bytes at 3.00369 s
Server received 1024 bytes at 4.00369 s
Server received 1024 bytes at 5.00369 s
Server received 1024 bytes at 6.00369 s
Server received 1024 bytes at 7.00369 s
Server received 1024 bytes at 8.00369 s
Server received 1024 bytes at 9.00369 s
Server received 1024 bytes at 10.0037 s
Server received 1024 bytes at 11.0037 s
```